41039 Programming 1

Revision of Week 5: Classes, objects and enums



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the revision session works

A recap before the next lecture

Write an answer before the reveal. Explain the state change, rule or contract that makes it follow. These are practice questions, with no new assessment requirements.

Part 1: on your own

- 8 short questions
- About 90 seconds per question
- Java and Python; name the language first

Part 2: teams of 3 or 4

- Draw the model
- Spot the errors
- Predict the output
- Write and test the code

A result needs a reason. A repair needs a counterexample.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

1. Class count and object count

Java 8; commit to an outcome before the reveal.

```
public class Counter {
    private static int total = 0;
    private int mine = 0;
    public void click() {
        total++;
        mine++;
    }
    public static void main(String[] args) {
        Counter a = new Counter();
        Counter b = new Counter();
        a.click();
        a.click();
        b.click();
        System.out.println(a.mine + " " + b.mine + " " +
      total);
    }
}
```

Your answer

What is the complete output? Which line or rule decides it?

1. Class count and object count

Java 8; commit to an outcome before the reveal.

```
public class Counter {
    private static int total = 0;
    private int mine = 0;
    public void click() {
        total++;
        mine++;
    }
    public static void main(String[] args) {
        Counter a = new Counter();
        Counter b = new Counter();
        a.click();
        a.click();
        b.click();
        System.out.println(a.mine + " " + b.mine + " " +
      total);
    }
}
```

Answer

```
2 1 3
```

A static field has exactly one copy, on the class, shared by every object ; so all three clicks count towards total. mine is an instance variable: each object has its own, so a counts 2 and b counts 1. Static fields are never copied into objects and never reset by a constructor.

2. A constructor that changes nothing

Java 8; commit to an outcome before the reveal.

```
public class P {
    private int x;
    public P(int x) {
        x = x;
    }
    public static void main(String[] args) {
        System.out.println(new P(5).x);
    }
}
```

Your answer

What is the complete output? Which line or rule decides it?

2. A constructor that changes nothing

Java 8; commit to an outcome before the reveal.

```
public class P {
    private int x;
    public P(int x) {
        x = x;
    }
    public static void main(String[] args) {
        System.out.println(new P(5).x);
    }
}
```

Answer

0

Inside the constructor, x means the parameter, which hides the field of the same name. So x = x copies the parameter onto itself, and the field keeps its default value, 0. It is legal (at most a compiler warning), which is exactly what makes it a nasty bug. this.x = x; is the fix ; the first use of this in section 4.3.

3. Two dogs, one list

Python 3; commit to an outcome before the reveal.

```
class Dog:
    tricks = []
    def __init__(self, name):
        self.name = name
    def add(self, trick):
        self.tricks.append(trick)
a = Dog("Rex")
b = Dog("Fido")
a.add("sit")
print(b.name, b.tricks)
```

Your answer

What is the complete output? Which line or rule decides it?

3. Two dogs, one list

Python 3; commit to an outcome before the reveal.

```
class Dog:
    tricks = []
    def __init__(self, name):
        self.name = name
    def add(self, trick):
        self.tricks.append(trick)
a = Dog("Rex")
b = Dog("Fido")
a.add("sit")
print(b.name, b.tricks)
```

Answer

```
Fido ['sit']
```

name is set in ___init___ through self, so each dog has its own. tricks is created once, in the class body: a class attribute, one list shared by every dog ; like a Java static field. append mutates that shared list. Put self.tricks = [] in ___init___ to give each dog its own.

4. The implicit class argument

Python 3; commit to an outcome before the reveal.

```
class Foo:
    baz = "blue"
    @classmethod
    def bar(cls):
        print("The class says " + cls.baz)
Foo.bar()
```

Your answer

What is the complete output? Which line or rule decides it?

4. The implicit class argument

Python 3; commit to an outcome before the reveal.

```
class Foo:
    baz = "blue"
    @classmethod
    def bar(cls):
        print("The class says " + cls.baz)
Foo.bar()
```

Answer

```
The class says blue
```

A @classmethod receives the class itself as its first parameter, conventionally named cls, and Python supplies it automatically ; you do not pass it. Through cls the method can read class attributes such as baz. It is called from the class, like a Java static method.

5. Enum identity

Java 8; commit to an outcome before the reveal.

```
public class Cards {
    enum Suit { HEARTS, SPADES, DIAMONDS, CLUBS; }
    public static void main(String[] args) {
        Suit a = Suit.HEARTS;
        Suit b = Suit.values()[0];
        System.out.println(a + " " + (a == b));
    }
}
```

Your answer

What is the complete output? Which line or rule decides it?

5. Enum identity

Java 8; commit to an outcome before the reveal.

```
public class Cards {
    enum Suit { HEARTS, SPADES, DIAMONDS, CLUBS; }
    public static void main(String[] args) {
        Suit a = Suit.HEARTS;
        Suit b = Suit.values()[0];
        System.out.println(a + " " + (a == b));
    }
}
```

Answer

```
HEARTS true
```

An enum value prints as its name. There is exactly one object per enum value ; no others can ever be created ; so every reference to HEARTS is a reference to the same object, and == is true. values() returns an ordinary array, so indexing it is fine.

6. An enum alias

Python 3; commit to an outcome before the reveal.

```
from enum import Enum
class Colour(Enum):
    RED = 1
    GREEN = 1
    BLUE = 2
print(Colour.GREEN)
print(len(Colour))
```

Your answer

What is the complete output? Which line or rule decides it?

6. An enum alias

Python 3; commit to an outcome before the reveal.

```
from enum import Enum
class Colour(Enum):
    RED = 1
    GREEN = 1
    BLUE = 2
print(Colour.GREEN)
print(len(Colour))
```

Answer

```
Colour.RED
2
```

GREEN and RED have the same value, so GREEN aliases RED. Printing the member uses its canonical name, Colour.RED. len(Colour) counts RED and BLUE, giving 2. The alias does not create a third member.

7. Switch expressions

Java 14 excerpt: main statements and supporting types; use Ed or a matching local JDK; commit to an outcome before the reveal.

```
int n = 7;
String s = switch (n % 3) {
    case 0 -> "fizz";
    case 1 -> "one";
    default -> "other";
};
System.out.println(s);
```

Your answer

What is the complete output? Which line or rule decides it?

7. Switch expressions

Java 14 excerpt: main statements and supporting types; use Ed or a matching local JDK; commit to an outcome before the reveal.

```
int n = 7;
String s = switch (n % 3) {
    case 0 -> "fizz";
    case 1 -> "one";
    default -> "other";
};
System.out.println(s);
```

Answer

```
one
```

7 % 3 is 1, so the case 1 arm gives "one". With the arrow form there is no fall-through: the switch produces the value after the arrow and is finished, so no break is needed. Because it produces a value, it can be assigned ; that is the whole point of a switch expression.

8. Guard order in a pattern switch

Java 21; use Ed or a matching local JDK; commit to an outcome before the reveal.

```
public class Demo {
record Student(String name, int mark) {}
static String grade(Object o) {
    return switch (o) {
        case Student s when s.mark() >= 50 -> "P";
        case Student s when s.mark() >= 85 -> "HD";
        case Student s -> "Z";
        default -> "not a student";
    };
}
 public static void main(String[] args) { System.out.
     println(grade(new Student("An",92))); }
}
```

Your answer

What is the complete output? Which line or rule decides it?

8. Guard order in a pattern switch

Java 21; use Ed or a matching local JDK; commit to an outcome before the reveal.

```
public class Demo {
record Student(String name, int mark) {}
static String grade(Object o) {
    return switch (o) {
        case Student s when s.mark() >= 50 -> "P";
        case Student s when s.mark() >= 85 -> "HD";
        case Student s -> "Z";
        default -> "not a student";
    };
}
 public static void main(String[] args) { System.out.
     println(grade(new Student("An",92))); }
}
```

Answer

```
P
```

Cases are tried in order, and the first whose pattern matches and whose when is true wins. 92 >= 50, so the first case answers "P" and the HD case is never reached ; the same ordering rule as an else if ladder in Weeks 2+3. (Put the >= 85 case first.) Guarded cases like these are allowed to overlap.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Form teams

- Teams of 3 or 4. One shared sheet, one spokesperson.
- For each challenge, record the result and the reason.
- Rotate the spokesperson. Other teams can challenge an explanation.
- The final task has starter files and a Java/Python solution pair.

Discuss first. Commit to an answer. Then compare with the reveal.

Draw it: Two names and two accounts

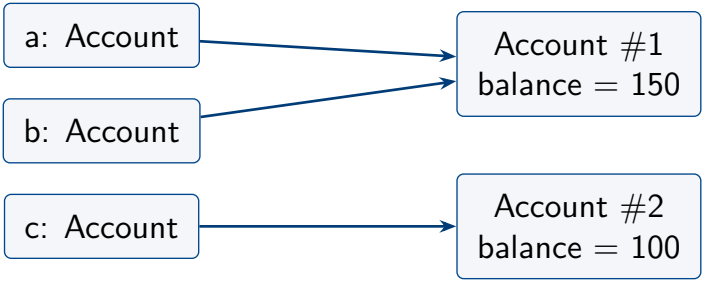
```
Account a=new Account(100);
Account b=a;
Account c=new Account(100);
b.deposit(50);
System.out.println(a.balance+" "+b.balance+" "+c.balance
    );
System.out.println((a==b)+" "+(a==c));
class Account {
    int balance;
    Account(int balance) { this.balance=balance; }
    void deposit(int amount) { balance+=amount; }
}
```

Course website: Week 5. Complete programs accompany this deck.

Four minutes

Draw the object references after the shown operations. Label the important state. Mark what is shared, what changes, or which path handles failure. Predict every output line.

Draw it: one answer



Output

```
150 150 100
true false
```

a and b share one object. c refers to a separately constructed account with its own balance.

Course website: Week 5. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Spot the errors 1: Java (5 min)

Contract

After depositing 50 into a, balances must be 150 and 200.

```java
public class Broken {
    public static void main(String[] args) {
        Account a=new Account(100);
        Account b=new Account(200);
        a.deposit(50);
        System.out.println(a.balance()+" "+b.balance());
    }
}
class Account {
    private static int cents;
    Account(int cents) { cents=cents; }
    void deposit(int amount) { cents=amount; }
    int balance() { return cents; }
}
```

Spot the errors 1: answers

The repairs

- **L10:** Use an instance field: different accounts need separate balances.
- **L11:** Assign the parameter to this.cents.
- **L12:** Add the deposit rather than replacing the balance.

Course website: Week 5. Complete programs accompany this deck.

✗ Original program

```
50 50
```

✓ After the repairs

```
150 200
```

Spot the errors 2: Python (5 min)

Contract

The accounts must have independent histories, and print(a) must produce 150.

```
1 class Account:
2     deposits=[]
3     def __init__(self, cents): self.cents=
       cents
4     def deposit(self, amount):
5         self.deposits.append(amount)
6         self.cents=amount
7     def __str__(self): print(self.cents)
```

```
8
9  a=Account(100)
10 b=Account(200)
11 a.deposit(50)
12 print(a.cents,b.cents)
13 print(b.deposits)
14 print(a)
```

Spot the errors 2: answers

The repairs

- **L2:** Allocate a fresh deposit list inside `__init__` for each account.
- **L6:** Add to the current balance instead of replacing it.
- **L7:** Return a string from `__str__`; printing inside it returns None.

Course website: Week 5. Complete programs accompany this deck.

✗ Original program

```
50 200
[50]
50
TypeError: __str__ returned non-
    string (type NoneType)
```

✓ After the repairs

```
150 200
[]
150
```

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Predict 1: Aliasing, then replacement

```
Box a=new Box(2);
Box b=a;
b.side=3;
System.out.println(a.volume());
b=new Box(4);
System.out.println(a.volume()+" "+b.volume());
class Box {
    int side;
    Box(int side) { this.side=side; }
    int volume() { return side*side*side; }
}
```

Course website: Week 5. Complete programs accompany this deck.

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Predict 1: answer

```
27
27 64
```

Why

The first assignment shares a box. The later constructor creates a new box for b; a still refers to the side-3 box.

Course website: Week 5. Complete programs accompany this deck.

Predict 2: A counter is not an identity

```
class Ticket:
    issued=0
    def __init__(self):
        Ticket.issued+=1
        self.number=Ticket.issued
a,b=Ticket(),Ticket()
c=a
c.number=9
print(a.number,b.number,Ticket.issued)
print(a is c,a is b)
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 5. Complete programs accompany this deck.

Predict 2: answer

```
9 2 2
True False
```

Why

issued counts constructor calls. c aliases a, so changing c.number changes a.number without creating a third ticket.

Course website: Week 5. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Code it: specification

A bank account with a precise contract

Read the initial non-negative balance. Then process D amount or W amount until Q. Refuse non-positive amounts and withdrawals above the balance. Print OK or Refused for every command, then Balance: N. Use integer cents.

Example input

```
1000
D 500
W 200
Q
```

Expected output

```
OK
OK
Balance: 1300
```

Eight minutes. One Java team and one Python team at the keyboard. Other teams produce a boundary test and explain which wrong implementation it would expose.

Code it: Java solution (1/3)

```java
import java.util.Scanner;
public class Bank {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount acc = new BankAccount(sc.nextInt());
        String command = sc.next();
        while (!command.equals("Q")) {
            int amount = sc.nextInt();
            boolean ok;
            if (command.equals("D")) {
                ok = acc.deposit(amount);
            } else {
                ok = acc.withdraw(amount);
            }
            System.out.println(ok ? "OK" : "Refused");
```

Course website: Week 5. Complete programs accompany this deck.

Code it: Java solution (2/3)

```
16              command = sc.next();
17          }
18          System.out.println("Balance: " + acc.getBalance());
19      }
20  }
21  class BankAccount {
22      private int balance;
23      public BankAccount(int startingBalance) {
24          this.balance = startingBalance;
25      }
26      public boolean deposit(int cents) {
27          if (cents <= 0) {
28              return false;
29          }
30          this.balance += cents;
```

Course website: Week 5. Complete programs accompany this deck.

Code it: Java solution (3/3)

```
31          return true;
32      }
33      public boolean withdraw(int cents) {
34          if (cents <= 0 || cents > this.balance) {
35              return false;
36          }
37          this.balance -= cents;
38          return true;
39      }
40      public int getBalance() {
41          return this.balance;
42      }
43  }
```

Course website: Week 5. Complete programs accompany this deck.

Code it: Python solution (1/2)

```python
class BankAccount:
    def __init__(self, starting):
        self.__balance = starting
    def deposit(self, cents):
        if cents <= 0:
            return False
        self.__balance += cents
        return True
    def withdraw(self, cents):
        if cents <= 0 or cents > self.__balance:
            return False
        self.__balance -= cents
        return True
    def balance(self):
        return self.__balance
```

Course website: Week 5. Complete programs accompany this deck.

Code it: Python solution (2/2)

```
acc = BankAccount(int(input()))
line = input()
while line != "Q":
    parts = line.split()
    amount = int(parts[1])
    if parts[0] == "D":
        ok = acc.deposit(amount)
    else:
        ok = acc.withdraw(amount)
    if ok:
        print("OK")
    else:
        print("Refused")
    line = input()
print("Balance: " + str(acc.balance()))
```

Course website: Week 5. Complete programs accompany this deck.

Code it: the tests

A slash separates input lines; (blank) denotes an empty line.

Test 1
`1000 / D 500 / W 200 / Q`
Output: `OK / OK / Balance: 1300`

Test 2
`100 / W 500 / D -5 / D 50 / Q`
Output: `Refused / Refused / OK / Balance: 150`

Test 3
`0 / Q`
Output: `Balance: 0`

Test 4
`250 / W 250 / W 1 / D 0 / Q`
Output: `OK / Refused / Refused / Balance: 0`

Explain why each test is present, not only whether it passes.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

What you should now be able to explain

- Distinguish class state from instance state.
- Trace references separately from objects.
- Use constructors, this/self, enums and string representations.
- Explain switch-expression values and the order of guarded cases.

Return to the matching week page for runnable examples and further practice. Bring one unresolved prediction or counterexample to the next class.

Course website: Week 5. Complete programs accompany this deck.